

I.E.S Al-Mudeyne

STUDYHUB



Grado Superior de Desarrollo de Aplicaciones Web

Realizado por:

Manuel Cándido Mendoza González

Departamento:

Informática y comunicaciones

Los Palacios y Villafranca, Junio 2025



INDICE

1. Introducción
2. Estudio de Viabilidad
 - 2.1. Descripción del Sistema Actual
 - 2.2. Descripción del Sistema Nuevo
 - 2.3. Identificación de Requisitos del Sistema
 - 2.3.1. Requisitos de información
 - 2.3.2. Requisitos funcionales.
 - 2.3.3. Otros Requisitos
 - 2.4. Descripción de la solución
 - 2.5. Planificación del proyecto
 - 2.5.1. Equipo de trabajo
 - 2.5.2. Planificación temporal
 - 2.6. Estudio del coste el proyecto
3. Análisis del Sistema de Información
 - 3.1. Identificación del entorno tecnológico
 - 3.2. Modelado de datos
 - 3.2.1. Modelo Entidad-Relación
 - 3.2.2. Esquema de la base de datos
 - 3.3. Identificación de los usuarios participantes y finales
 - 3.4. Diagramas de Análisis
 - 3.5. Definición de interfaces de usuario
 - 3.5.1. Especificación de principios generales de interfaz
 - 3.5.2. Especificación de formatos individuales de la interfaz de pantalla
 - 3.5.3. Identificación de perfiles de usuario
 - 3.5.4. Especificación de formatos de impresión
 - 3.5.5. Especificación de la navegabilidad entre pantallas
4. Construcción del Sistema
 - 4.1. GitHub
 - 4.2. Docker
5. Conclusiones
6. Glosario de términos

7. Bibliografía

Apéndices

CARGOS EN EL PROYECTO.

Nombre:	
Cargo:	
Email:	
Función:	

REQUISITOS DE INFORMACIÓN

[RI_01]	
Descripción:	
Lista de campos:	
Observaciones:	

REQUISITOS FUNCIONALES

[RF_01]	
Descripción:	
Observaciones:	

OTROS REQUISITOS.

[RNF_01]	
Descripción:	
Observaciones:	

DIAGRAMA DE CASOS DE USO.

Actores

AC-01	
Descripción:	

Casos de Uso

CU-01		
Precondición		
Descripción		
Secuencia normal	Paso	Acción
	1	
	2	
	3	
Postcondición		
Comentarios		

1. Introducción

StudyHub-App nace como una solución integral para la gestión, comercialización y realización de cursos online. En un entorno digital donde la formación continua es esencial, esta plataforma ofrece un ecosistema completo tanto para estudiantes como para instructores.

El proyecto se define como una aplicación web que permite a los usuarios:

- **Aprender:** Accediendo a un marketplace diverso con cursos de distintas temáticas.
- **Enseñar:** Proporcionando herramientas intuitivas para que cualquier experto pueda crear, estructurar y publicar su propio contenido educativo.
- **Gestionar:** Mediante un panel de administración avanzado que garantiza la calidad del contenido y la correcta gestión de los usuarios.

Desarrollada con tecnologías como Laravel y Tailwind CSS, StudyHub-App prioriza la experiencia de usuario (UX), la seguridad y la escalabilidad.

2. Estudio de Viabilidad

2.1. Descripción del Sistema Actual

En la actualidad, muchas plataformas de formación online presentan barreras de entrada significativas para nuevos creadores (comisiones altas, interfaces complejas) o carecen de una integración fluida entre la creación de contenido y la gestión administrativa. Muchos procesos de validación de cursos se realizan de forma manual y desorganizada, lo que retrasa la disponibilidad de contenido para los alumnos.

2.2. Descripción del Sistema Nuevo

StudyHub-App propone un sistema centralizado que optimiza el flujo de trabajo educativo:

- **Flujo Guiado:** Un sistema de creación de lecciones en pocos pasos que facilita la publicación de contenido.
- **Seguridad y Validación:** Los cursos nuevos requieren la aprobación de un administrador antes de ser públicos, asegurando estándares de calidad.
- **Automatización:** Generación automática de certificados en PDF y seguimiento de progreso en tiempo real sin intervención manual.
- **Modernidad:** Interfaz adaptable con soporte nativo para modo oscuro y diseño orientado a la simplicidad.
- **Gratuito:** Capacidad para poder realizar tanto como usuario o como experto un curso de manera TOTALMENTE GRATUITA.

2.3. Identificación de Requisitos del Sistema

2.3.1. Requisitos de información

El sistema debe gestionar y almacenar de forma segura los siguientes conjuntos de datos:

- **Usuarios:** Perfiles, credenciales, roles y preferencias.
- **Académico:** Cursos (descripciones, precios, estados), categorías y lecciones multimedia (vídeos, PDFs, texto enriquecido).
- **Progreso:** Registro de lecciones completadas, cursos en curso y certificados emitidos.
- **Financiero:** Información de pago e historial de transacciones.

2.3.2. Requisitos funcionales

- **Gestión de Cuentas:** Registro, login, recuperación de contraseña y gestión de perfiles.
- **Marketplace:** Búsqueda avanzada, filtrado por categorías y sistema de compra de cursos.
- **Aprendizaje:** Reproductor de lecciones compatible con múltiples formatos (Video, PDF, Editor.js).
- **Autoría:** Herramientas para la creación, edición y activación de cursos por parte de los instructores.
- **Administración:** Panel de control para la gestión de usuarios, validación de cursos y mantenimiento de roles/permisos.

2.3.3. Otros Requisitos (No funcionales)

- Rendimiento: Carga rápida de assets mediante el uso de Vite.
- Seguridad: Protección contra vulnerabilidades comunes (CSRF, SQL Injection) y gestión granular de permisos con Spatie.
- Portabilidad: Despliegue simplificado mediante contenedores Docker.
- Usabilidad: Diseño responsive que garantice una experiencia óptima en móviles y escritorio.

2.4. Descripción de la solución

La solución técnica se basa en una arquitectura **MVC (Modelo-Vista-Controlador)** reforzada con una **capa de servicios** para desacoplar la lógica de negocio de los controladores. Se utiliza **Laravel** como núcleo del backend por su robustez, y un frontend dinámico construido con **Blade, Tailwind CSS y Alpine.js**. La persistencia de datos se gestiona con **MySQL** y el entorno de desarrollo/producción se estandariza mediante **Docker**.

2.5. Planificación del Proyecto

2.5.1. Equipo de Trabajo

Los componentes encargados de la ejecución y supervisión de este proyecto son:

Tabla: Componentes del proyecto I

Nombre:	Ramón Morales Pacheco
Cargo:	Project Manager / Product Owner
Email:	rmorpac932@g.educaand.es
Función:	Determina las expectativas del producto, valida los requisitos de negocio y asegura el cumplimiento de los plazos. Supervisa la calidad técnica y prioriza el backlog.

Tabla: Componentes del proyecto II

Nombre:	Manuel Cándido Mendoza González
Cargo:	Lead Full-stack Developer
Email:	Mmengon2204@g.educaand.es
Función:	Único responsable de la ejecución técnica. Realiza el análisis, diseño de base de datos, arquitectura de servicios, desarrollo frontend/backend, gestión de infraestructura (Docker) y pruebas.

2.5.2. Planificación Temporal

Para la realización de este proyecto se ha utilizado la metodología **SCRUM**, permitiendo un desarrollo iterativo e incremental. Se han definido los siguientes artefactos:

- **Product Backlog:** Lista priorizada de funcionalidades (Marketplace, Creación de cursos, Certificados, etc.).

- **Sprint Backlog:** Tareas específicas seleccionadas para cada ciclo de desarrollo.
- **Incremento del producto:** Versiones funcionales entregadas al finalizar cada sprint.

Tareas y temporalización estimada:

Tabla 37. Tareas y temporalización

TAREA	TIEMPO ESTIMADO
Análisis de requisitos y documentación inicial	2 semana
Diseño de Base de Datos y Modelo Entidad-Relación	1 semana
Creación del repositorio y configuración inicial	3 días
Sistema de Autenticación, Perfiles y Dashboard	3 semanas
Marketplace, Cursos y Capa de Servicios	4 semana
Lecciones multimedia, Editor.js y Progreso	2 semanas
Generación de Certificados PDF y Sistema de Pagos	1 semana
Dockerización y configuración de entornos (Local/Prod)	1 semana
Pruebas finales, depuración y cierre de documentación	1 semana

| **TOTAL** | 15 semanas y 3 días (~3,5 meses) |

Nota: Se estima que el proyecto incluye un total de **400 horas de trabajo efectivo, distribuidas en las tareas mencionadas anteriormente.*

2.6. Estudio del coste del proyecto

Los costes del proyecto se desglosan en los siguientes puntos, tomando como referencia el **Convenio Colectivo estatal de empresas de consultoría y estudios de mercado** (publicado en el BOE).

2.6.1. Costes de diseño y desarrollo

Se considera una jornada de trabajo proporcional a la duración del proyecto (3,5 meses) para cubrir los diferentes roles necesarios para la ejecución técnica por parte de Manuel (Desarrollo) y la supervisión de Ramón (Gestión).

Tabla: Costes de personal

RECURSO	CLASIFICACIÓN (Según convenio)	SUELDO BRUTO ANUAL (Según Convenio)	REPERCUSIÓN EN PROYECTO
Director de Proyecto / Product Owner** (Ramón)	Área 3 Grupo A Nivel 1	30.004,24 €	625,09 € *(0,25 meses FTE)
Jefe de Proyecto / Scrum Master (Ramón)	Área 3 Grupo B Nivel 1	29.105,06 €	606,35 € *(0,25 meses FTE)
Analista y backend developer (Manuel)	Área 3 Grupo C Nivel 1	27.595,48 €	2.299,62 € *(1,00 meses FTE)
Frontend Developer y UI/UX	Área 3 Grupo D Nivel 3	18.002,27 €	1.125,14 € *(0,75 meses FTE)

QA Tester y DevOps	Área 3 Grupo D Nivel 3	18.002,27 €	375,05 € *(0,25 meses FTE)
		TOTAL	5.031,25 €

Fuente: Elaboración propia a partir de BOE.

2.6.2. Hardware y software de desarrollo

El proyecto se ha desarrollado utilizando los equipos personales, sin necesidad de realizar inversiones en hardware. Las tecnologías empleadas son:

Tabla: Costes de software

SOFTWARE	COSTE
IDE Visual Studio Code	Gratuito
Docker / Docker Desktop	Gratuito
Laragon / Entorno local	Gratuito
Laravel Framework 10.x	Gratuito
MySQL 8.0	Gratuito
Git / GitHub	Gratuito
Postman (Testing de API)	Gratuito
TOTAL	0,00€

2.6.3. Costes de infraestructuras (Explotación)

Para el despliegue del sistema en producción, se ha optado por el uso de contenedores Docker, lo que simplifica el uso de infraestructura en la nube.

- **Servidor Cloud (Ej. AWS EC2 / DigitalOcean):** VPS básico para el clúster de Docker: ~15€/mes.

- **Servicios Externos (SMTP Mailpit/Mailgun, Almacenamiento S3 opcional):** ~15€/mes.

- **Coste total de explotación estimado:** 30€/mes (360,00 €/año).

2.6.4. Costes de mantenimiento

Los costes de mantenimiento cubren posibles actualizaciones de Laravel, parches de seguridad y resolución de incidencias. Se estima **1 mes** de trabajo al año por parte de un desarrollador.

2.6.5. Resumen de costes

Resumen final del presupuesto teórico del proyecto StudyHub.

Tabla: Costes de mantenimiento

BLOQUE	COSTE
Desarrollo	5.031,25 €
Software y Herramientas	0,00€
Explotación - 1er año	360,00 €

Mantenimiento – 1er año	1.500,19 €
TOTAL	6.891,44 €

3. Análisis del Sistema de Información

3.1. Identificación del entorno tecnológico

En este apartado se diferencia entre el entorno tecnológico que se utiliza en el desarrollo y la explotación del sistema, detallando las herramientas, lenguajes, frameworks y plataformas a emplear, en línea con las necesidades de una aplicación moderna.

Entorno de desarrollo:

En la fase de desarrollo de la aplicación se utilizan las siguientes herramientas y tecnologías:

- Lenguajes de Programación:

- **Backend:** Para desarrollar la lógica de negocio se utiliza PHP 8.2.
- **Frontend:** Para la interfaz y el cliente se utiliza JavaScript, HTML5 y CSS3.

- Frameworks y Librerías:

- **Backend:** Se establece el uso del framework **Laravel 10.x**, ideal para una arquitectura robusta MVC y el desarrollo de servicios.
- **Frontend:** Se emplea **Tailwind CSS 3.x** para conseguir un diseño responsive, moderno y con soporte de modo oscuro. Se acompaña de **Alpine.js** para dotar de interactividad a componentes específicos sin necesidad de frameworks más pesados.

- Entorno de Desarrollo (IDE):

- El desarrollo completo de frontend y backend se ha realizado utilizando **Visual Studio Code**.

- Base de Datos:

- La persistencia de datos se apoya en **MySQL 8.0**, gestionada localmente a través de entornos como Laragon o contenedores.

- **Gestión de Dependencias y Compilación:**

- Backend: **Composer** gestiona las librerías PHP.
- Frontend: **NPM** administra los paquetes JS, y **Vite** se encarga de empaquetar y compilar los assets (HMR) logrando una carga ultrarrápida.

- **Control de Versiones:**

- Se realiza el control de versiones con **Git** y se usa un repositorio de **GitHub*** para almacenar el código y estructurar las ramas (main, develop).

Entorno de Explotación:

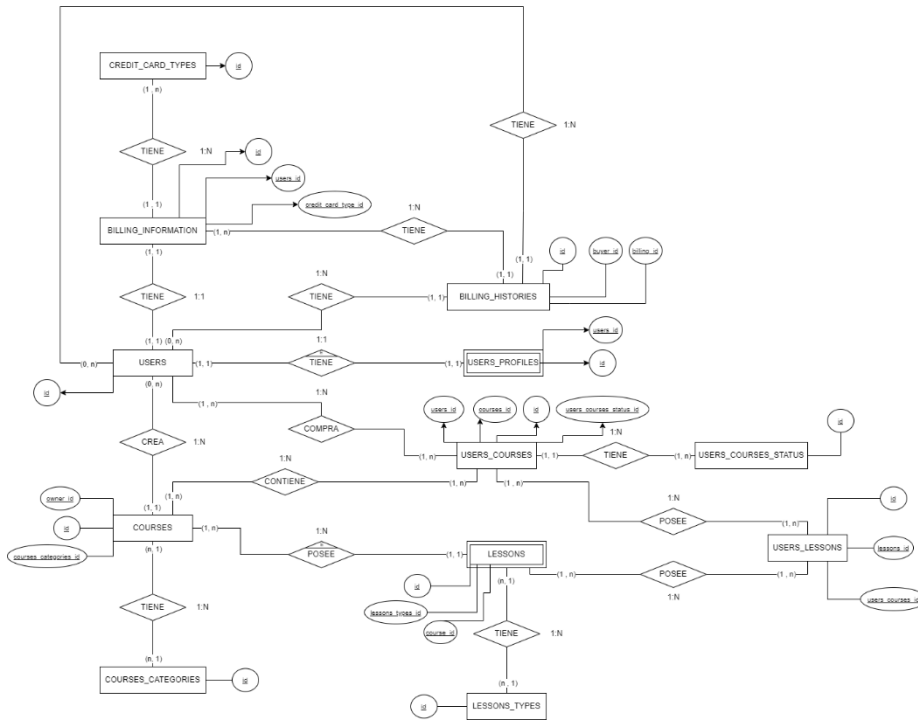
- **Plataforma de despliegue:** La aplicación se despliega encapsulando cada parte del sistema en entornos aislados mediante **Docker**. Se utiliza **Docker Compose** como orquestador para levantar y gestionar todos los servicios de forma sincronizada.
- **Contenedores:** El archivo `docker-compose.yml` levanta varios servicios independientes:
 - **App:** Contenedor con Apache y PHP para servir Laravel.
 - **DB:** Contenedor de MySQL con la persistencia configurada mediante volúmenes.
 - **phpMyAdmin*** Contenedor opcional para gestión visual de la BD en el puerto 8080.
 - **Mailpit:** Servidor de captura de correos electrónicos para el entorno de pruebas, accesible en el puerto 8025.

3.2. Modelado de datos

El modelado de datos refleja la arquitectura necesaria para sostener las entidades del sistema (usuarios, cursos, lecciones, pagos). Al estar desarrollada en Laravel, la base de datos se genera dinámicamente mediante el sistema de migraciones, que define tanto la estructura como las claves foráneas que garantizan la integridad referencial. Además, se emplea el ORM *Eloquent* para mapear estas tablas en modelos orientados a objetos.

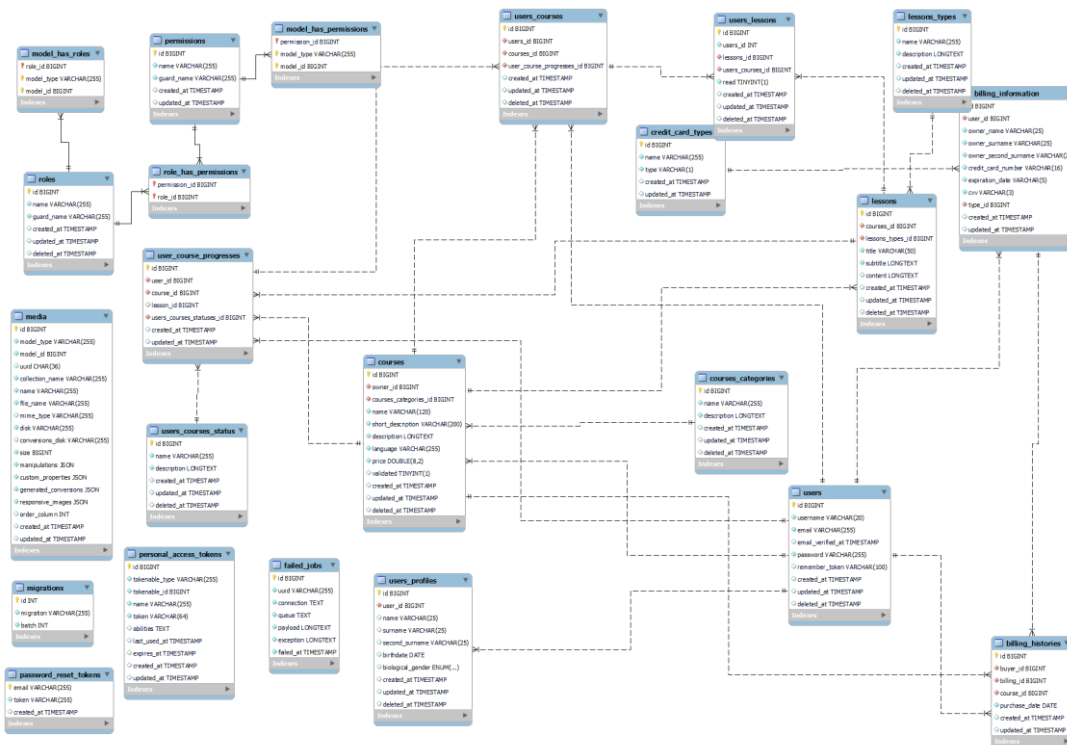
3.2.1. Modelo Entidad-Relación

El diagrama conceptual representa las entidades y cómo interactúan dentro de la plataforma (por ejemplo, la relación N:M entre `Users` y `Courses` que se resuelve en la tabla pivote de compras/progreso).



3.2.2. Esquema de la base de datos (importante)

El esquema relacional final, donde se aprecian las claves primarias (PK), las claves foráneas (FK) y la tipología de cada campo, se muestra en el siguiente diagrama estructural completo de la aplicación:



3.3. Identificación de los usuarios participantes y finales

Se detallan en este apartado los tipos de usuarios que están involucrados en el proceso de creación y uso de la aplicación, diferenciando entre participantes (desarrollo) y finales (explotación).

Usuarios participantes:

Son las personas involucradas directamente en el análisis, diseño, implementación y supervisión técnica del proyecto:

- **Desarrollador Principal (Manuel Cándido Mendoza González):** Responsable de la creación íntegra de la aplicación (frontend, backend, diseño y bases de datos), pruebas y documentación técnica.
- **Product Owner (Ramón):** Responsable de la validación técnica y académica del proyecto, orientando los objetivos y asegurando la calidad del desarrollo.

Usuarios finales:

Son las personas que utilizarán la plataforma de StudyHub de forma habitual. Al tratarse de una plataforma integral, los roles se definen según la interacción con los cursos:

- **Administrador:** Usuario con acceso total al panel de gestión (Dashboard de administración). Se encarga de moderar y validar los cursos de nueva creación, gestionar categorías, roles y realizar mantenimiento sobre las cuentas de usuario.
- **Usuario Estándar (Alumno / Instructor):** Cualquier usuario registrado en la plataforma. Tiene una doble vertiente:
 - Como **Alumno:** Puede explorar el marketplace, comprar cursos, consumir lecciones (vídeo, texto, PDF, editor) y descargar sus certificados de finalización y recibos.
 - Como **Instructor:** Puede diseñar y publicar sus propios cursos mediante el editor de lecciones. Deberá esperar la validación del administrador para que su contenido sea público en el marketplace.

3.4. Diagramas de Análisis

3.4.1. Casos de uso

Los casos de uso principales definen las interacciones esenciales entre los actores (Usuarios y Administradores) y el sistema. A continuación se resumen los flujos más relevantes:

CU-01: Creación y Validación de Cursos

- **Actor:** Usuario (creador) y Administrador.
- **Descripción:** Un usuario crea un curso y añade lecciones. El curso queda en estado "inactivo". El Administrador revisa el curso desde su panel y aprueba su publicación, haciéndolo visible en el marketplace.

CU-02: Compra y Realización de Curso

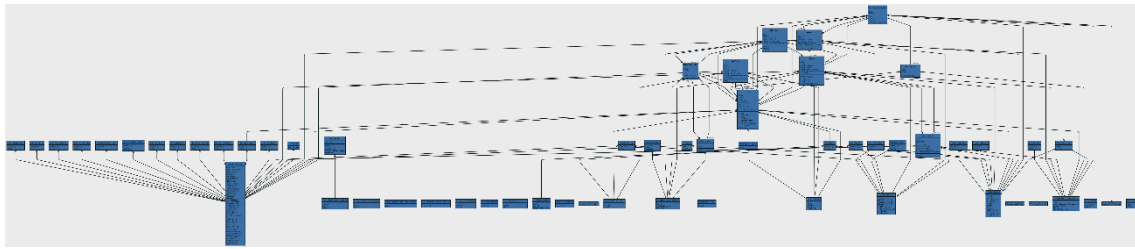
- **Actor:** Usuario (alumno).
- **Descripción:** El usuario filtra cursos en el marketplace, selecciona uno y realiza la compra (requiere tarjeta vinculada). El curso se añade a "Mis cursos". El usuario visualiza el contenido y, al llegar al 100% de progreso, desbloquea el certificado en PDF.

CU-03: Gestión Integral del Sistema

- **Actor:** Administrador.
- **Descripción:** El administrador puede aplicar un CRUD completo sobre cualquier entidad del sistema (borrado lógico de cursos/usuarios, edición de lecciones, asignación de permisos a roles).

3.4.2. Diagramas de Clases

El siguiente diagrama UML muestra la estructura de clases del sistema implementado en Laravel, donde los controladores delegan responsabilidades a la capa de servicios, comunicándose con los modelos gestionados a través de Eloquent ORM.



3.5. Definición de interfaces de usuario

3.5.1. Especificación de principios generales de interfaz

Para el diseño de StudyHub se han priorizado los estándares modernos de desarrollo web, enfocándose en ofrecer un producto SaaS (Software as a Service) de apariencia premium:

- **Estructura y Consistencia:** Se utiliza un *Layout* general compartido (app.blade.php / guest.blade.php). Los menús de navegación se sitúan en la barra superior o en un sidebar lateral para el panel de administración.
- **Estilos y Colores:** Construido sobre **Tailwind CSS**, se hace uso de una paleta neutra para los fondos y colores primarios vibrantes (azul/púrpura) para los elementos de acción (CTAs).
- **Modo Oscuro (Dark Mode):** Se ha implementado un soporte nativo de modo oscuro mediante las clases `dark:` de Tailwind, mejorando la ergonomía visual en entornos de baja luz.
- **Tipografía:** Se opta por fuentes sans-serif modernas y legibles en cualquier resolución, favoreciendo la lectura de las lecciones.

3.5.2. Especificación de formatos individuales de la interfaz de pantalla

El frontend proporciona distintas vistas según el flujo del usuario. Algunos de los formatos clave son:

- **Marketplace:** Un listado de cursos en formato de cuadrícula (grid) con tarjetas (cards) que muestran imagen, título, instructor y precio. Incorpora una barra de búsqueda y filtros por categoría.

- **Panel "Mis Cursos":** Distingue visualmente mediante pestañas los "Cursos Comprados" (mostrando una barra de progreso) y los "Cursos Creados" (mostrando el estado de validación).
- **Reproductor de Lecciones:** Una vista enfocada en el contenido. Muestra el visualizador multimedia (vídeo, visor PDF integrado o Editor.js) en el centro, y un índice navegable en el lateral.
- **Dashboard de Administración:** Pantalla enriquecida con tablas de datos que permiten búsquedas en tiempo real, ordenación y paginación para agilizar la moderación del sitio.

3.5.3. Identificación de perfiles de usuario

La interfaz cambia drásticamente en función del rol mediante validaciones con Spatie Permission:

- **Vista de Administrador:** Desbloquea rutas protegidas (`/admin`) con una disposición enfocada en datos crudos, estadísticas de uso y paneles de control.
- **Vista de Usuario:** Entorno enfocado en el aprendizaje y la venta cruzada (cross-selling). Rutas protegidas (`/dashboard`, `/profile`) pero con una interfaz más gráfica y amigable.

3.5.4. Especificación de formatos de impresión

El sistema es capaz de emitir documentos dinámicos e inmutables generados mediante la librería DomPDF:

- **Certificado de Finalización:** Un documento en formato **A4 horizontal**. Contiene el nombre del alumno, título del curso, fecha de emisión y un diseño con orlas de prestigio que certifica la finalización al 100%.
- **Recibo de Facturación (Billing):** Un documento en formato **A4 vertical**. Detalla la información transaccional de una compra (datos del usuario, nombre del curso adquirido, últimos 4 dígitos de la tarjeta utilizada y fecha).

3.5.5. Especificación de la navegabilidad entre pantallas

La estructura de rutas se divide en tres flujos principales:

1. **Flujo de Acceso:** `Landing Page` → `Login` / `Registro` → `Dashboard` (Vista principal del usuario autenticado).

2. **Flujo Comercial y de Aprendizaje:** `Dashboard` → `Marketplace` → `Detalle del Curso` → `Compra` → `Mis Cursos` → `Reproductor de Lecciones` → `Descarga de Certificado`.

3. **Flujo Administrativo:** `Dashboard` → Acceso al `Panel Administrador` (solo roles autorizados) → `Gestión de Cursos` / `Validación` / `Gestión de Usuarios` / `Categorías`.

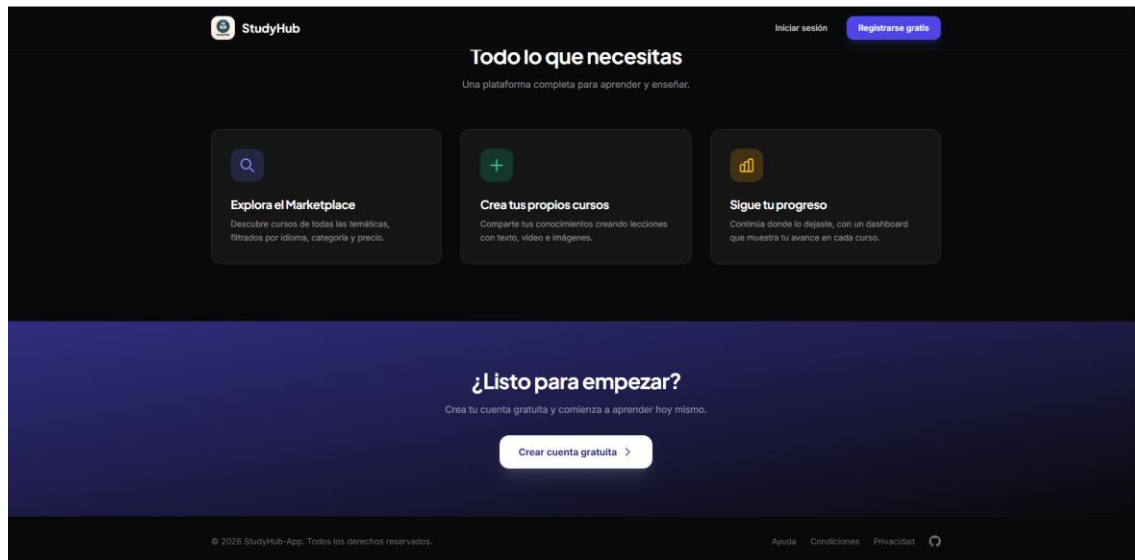
CAPTURAS STUDYHUB:

Todas las vistas están disponibles para escritorio y móvil. Se adjuntan las mas relevantes.

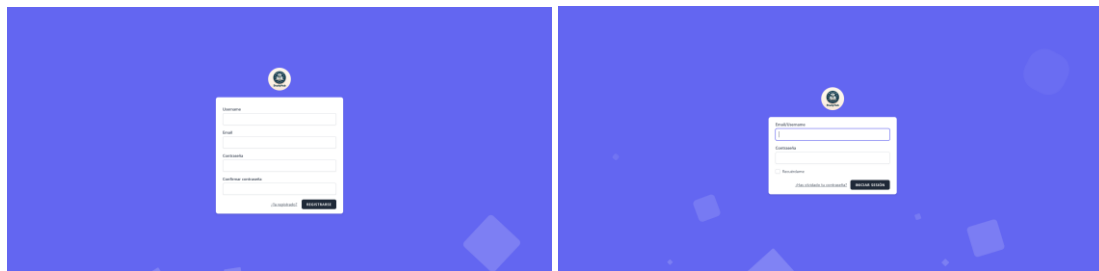
LANDPAGE:



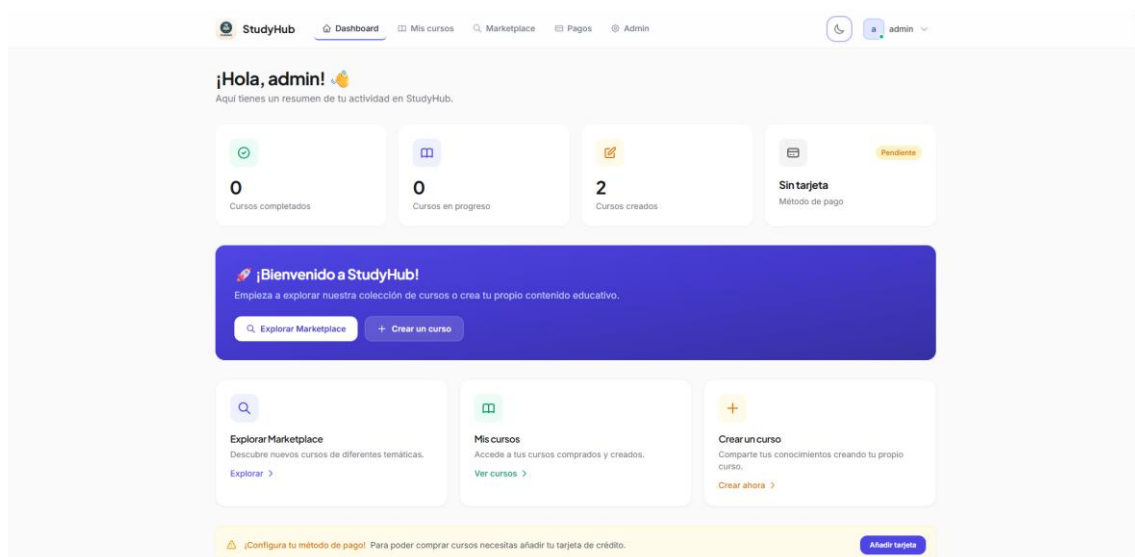
← Ejemplo vista móvil



REGISTRO/INICIO DE SESION:

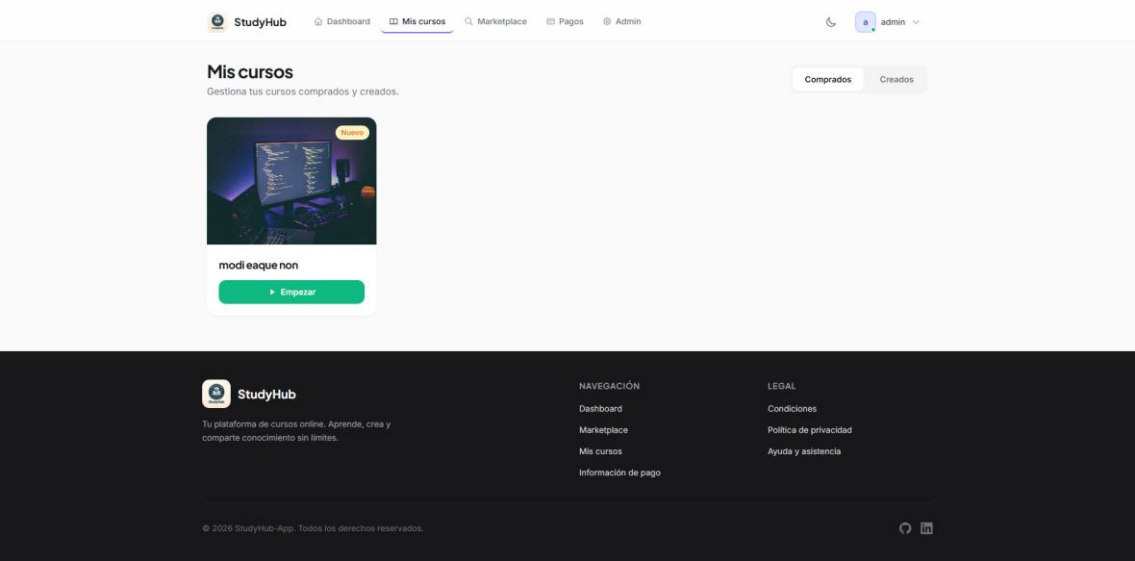


DASHBOARD:

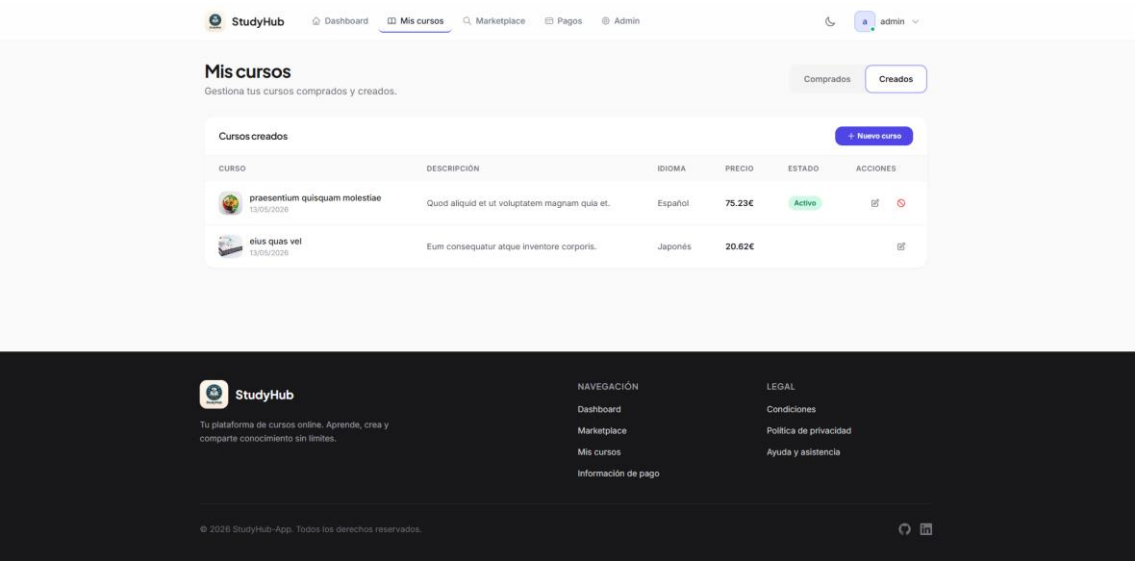


MIS CURSOS:

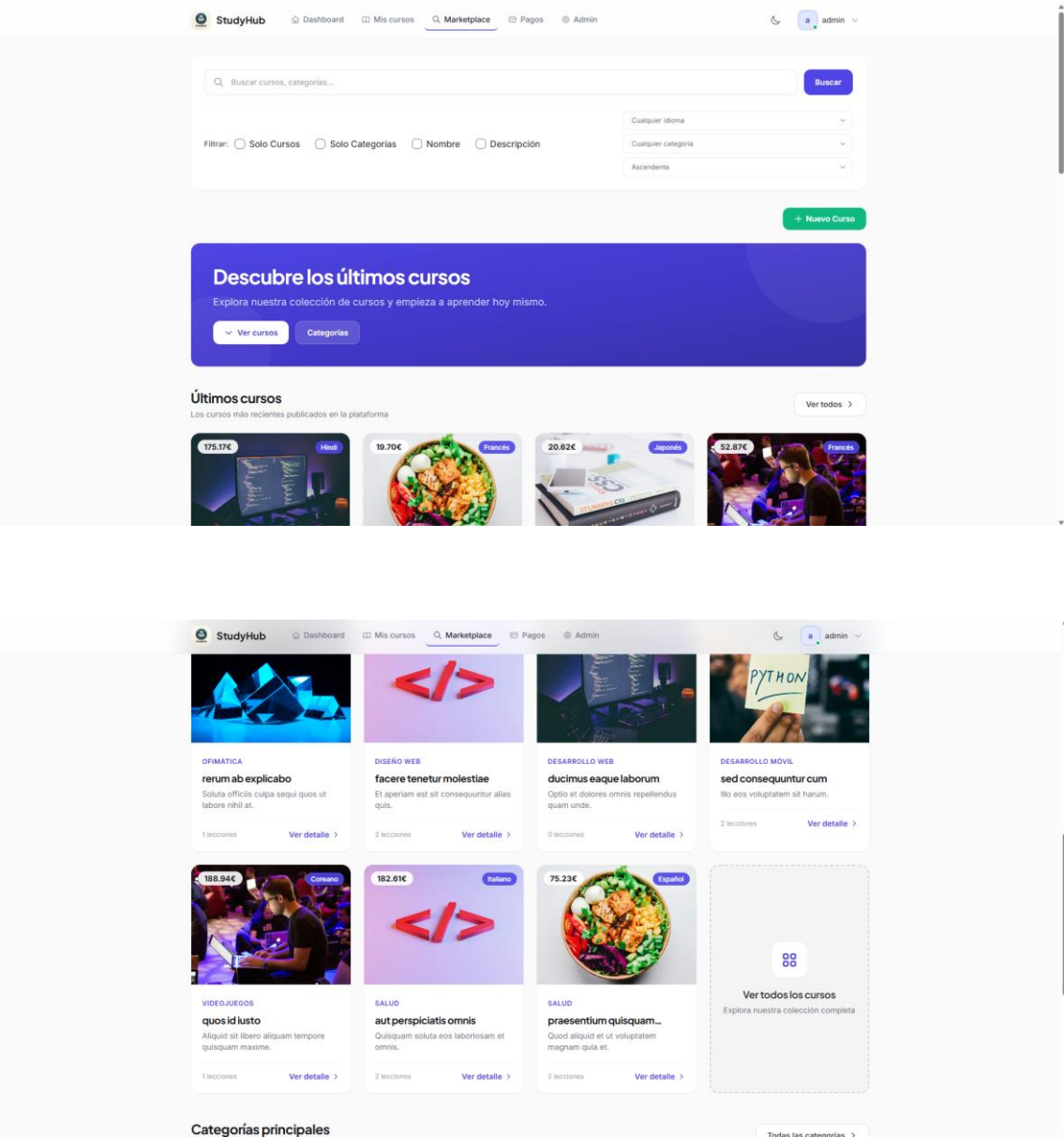
Comprados:



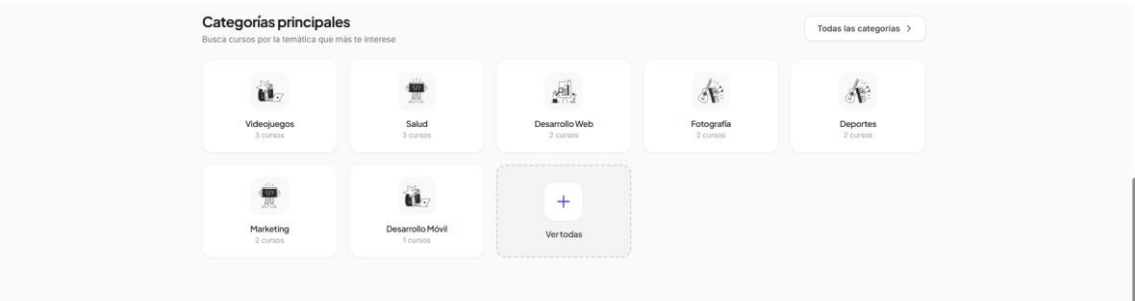
Creados:



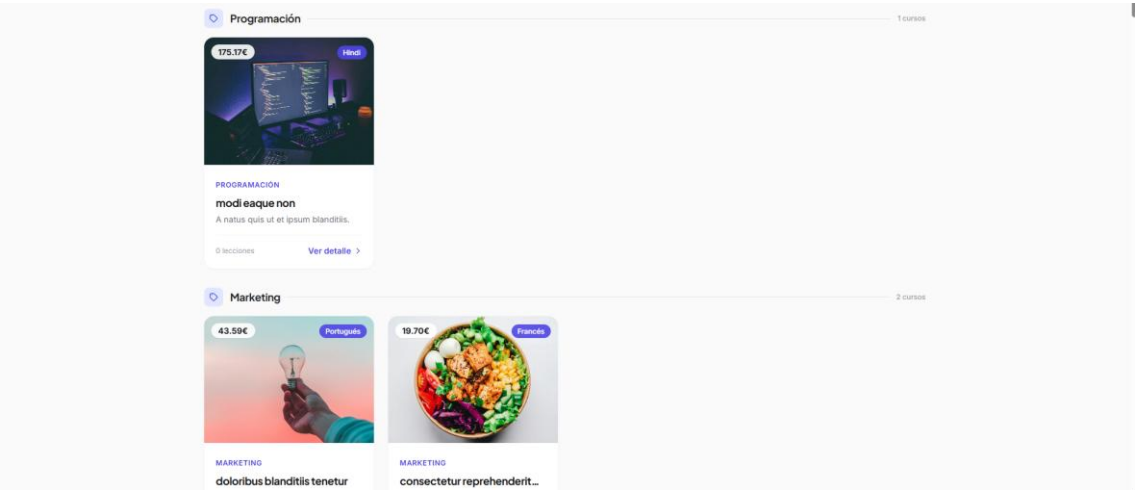
MARKETPLACE:



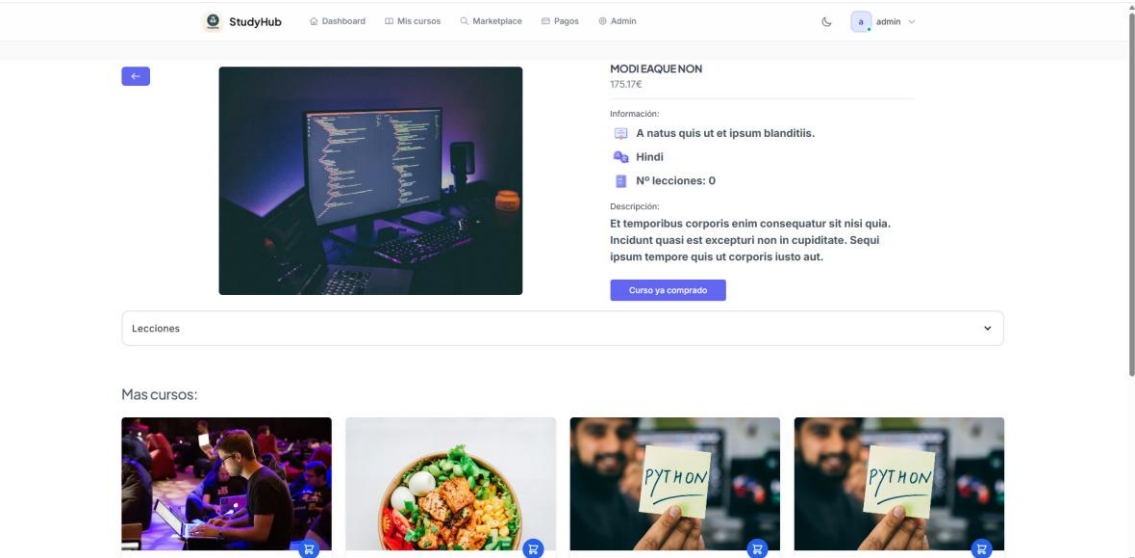
CATEGORÍAS:



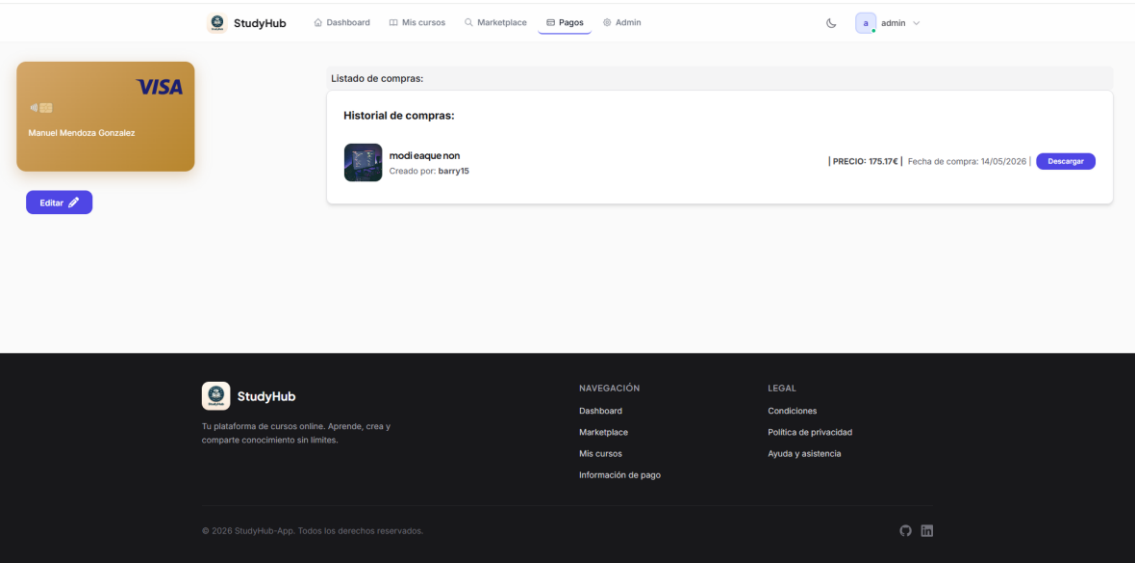
CURSOS POR CATEGORÍA:



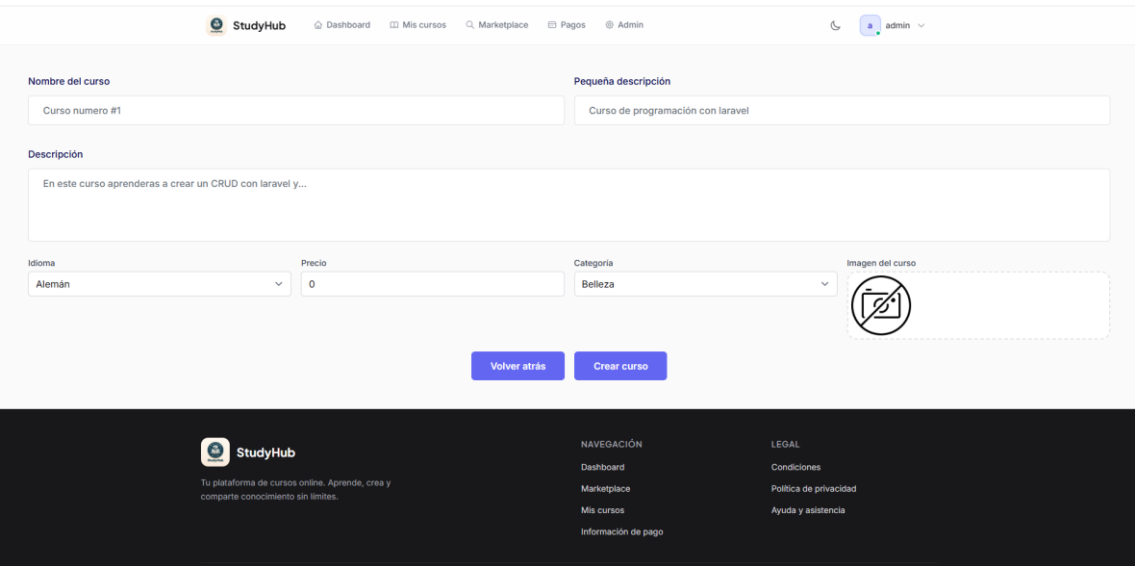
PÁGINA COMPRAR CURSO:



MÉTODO DE PAGO E HISTORIAL DE COMPRAS:



CREACIÓN DE CURSOS:



CREACIÓN DE LECCIONES:

StudyHub

Dashboard

Mis cursos

Marketplace

Pagos

Admin

admin

Título de la lección

Lección #1

Subtítulo

Programación orientada a objetos

Volver atrás

Siguiente

StudyHub

Dashboard

Mis cursos

Marketplace

Pagos

Admin

admin

Tipo de contenido

Personalizado

Contenido de la lección

asdfasdfsdf

// hola

+

⋮

in

PERFIL DE USUARIO:

Información de inicio de sesión

Puedes cambiar tu username y correo electrónico.

Username

admin

Email

admin@admin.com

GUARDAR

Información del perfil

Actualiza tu información personal.

Avatar



Nombre

AdminName

Primer apellido

AdminSurname

Segundo apellido

AdminSecondSurname

Cumpleaños

01/01/2024

Género biológico

Masculino

GUARDAR

Cambiar contraseña

Por seguridad tendrás que volver a iniciar sesión una vez cambiada la contraseña.

Contraseña actual

Nueva contraseña

Confirmar contraseña

GUARDAR

Suspender cuenta

Una vez que se suspenda su cuenta, ésta quedará inactiva y no podrá acceder a la información almacenada en ella. Para reactivarla deberá contactar con el administrador de la aplicación.

SUSPENDER CUENTA

Eliminar cuenta

Una vez que se elimine su cuenta, todos sus recursos y datos se eliminarán permanentemente. Antes de eliminar su cuenta, descargue cualquier dato o información que desee conservar.

ELIMINAR CUENTA

PANEL ADMINISTRADOR:

StudyHub

Dashboard

Mis cursos

Marketplace

Pagos

Admin

admin

Panel de Administración

Gestiona usuarios, cursos, categorías y roles de la plataforma.

Usuarios

Gestiona cuentas, roles y permisos de usuarios.

Gestionar >

Cursos

Crea, edita y valida los cursos de la plataforma.

Gestionar >

Categorías

Administra las categorías de los cursos.

Gestionar >

Roles

Configura los roles y permisos del sistema.

Gestionar >

Panel de Administración

Gestiona usuarios, cursos, categorías y roles de la plataforma.

Usuarios

Gestiona cuentas, roles y permisos de usuarios.

Gestionar >

Cursos

Crea, edita y valida los cursos de la plataforma.

Gestionar >

Categorías

Administra las categorías de los cursos.

Gestionar >

Roles

Configura los roles y permisos del sistema.

Gestionar >

LISTADO DE USUARIOS

Buscar...

Buscar

NUEVO USUARIO

Filtrar por:

Username

Email

Nombre

Primer Apellido

Segundo Apellido

dd/mm/aaaa

Todos

Todos

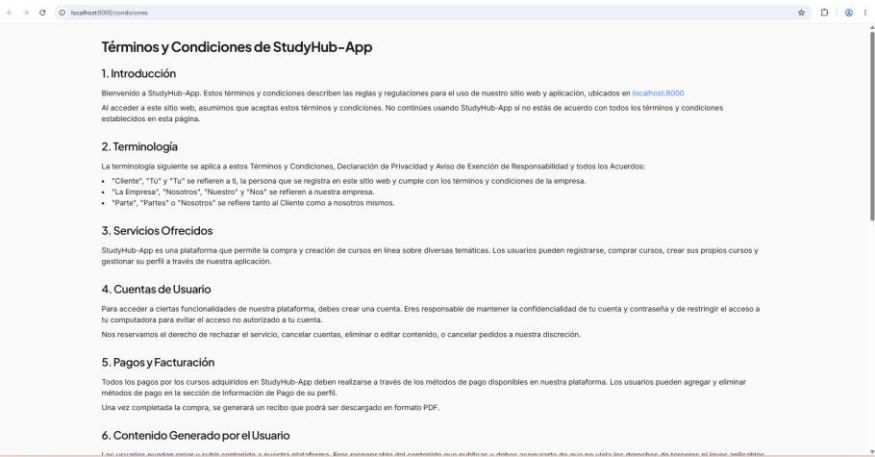
Todos

Ascendente

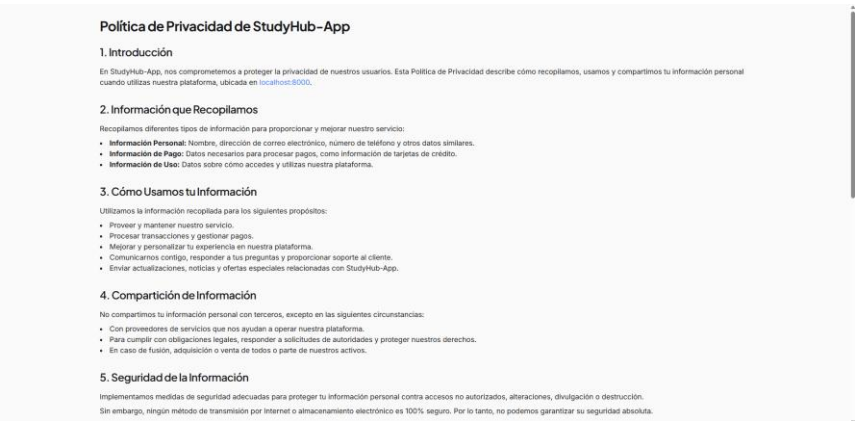
USERNAME	EMAIL	NOMBRE	PRIMER APELLIDO	SEGUNDO APELLIDO	FECHA DE NACIMIENTO	GÉNERO BIOLÓGICO	ROL	STATUS	OPCIONES
bkiehn	jamie.bednar@example.org	Howell Buckridge	Collier	Corwin	11-12-2000	Otro	Sin Rol	ACTIVO	
lueilwiz	rodriguez.hobart@example.org	Coy Witting	Kirlin	Trantow	03-05-1992	Masculino	Sin Rol	ACTIVO	
daugherty.izabella	xbednar@example.net	Jena Hessel DVM	Thiel	Bernier	13-12-1976	Femenino	Sin Rol	ACTIVO	

Página 29 de 36

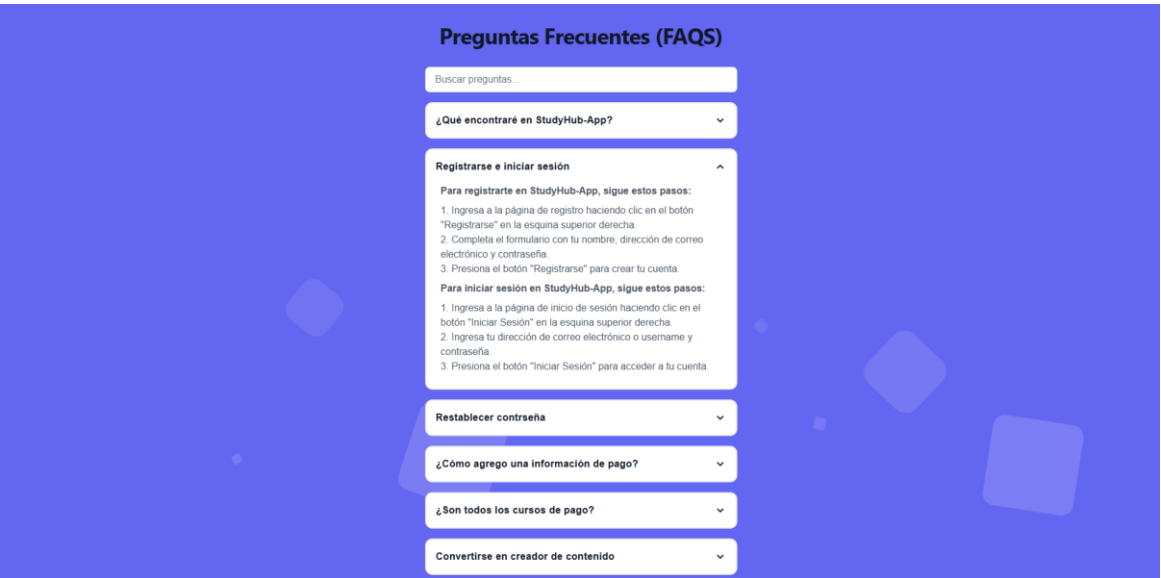
CONDICIONES:



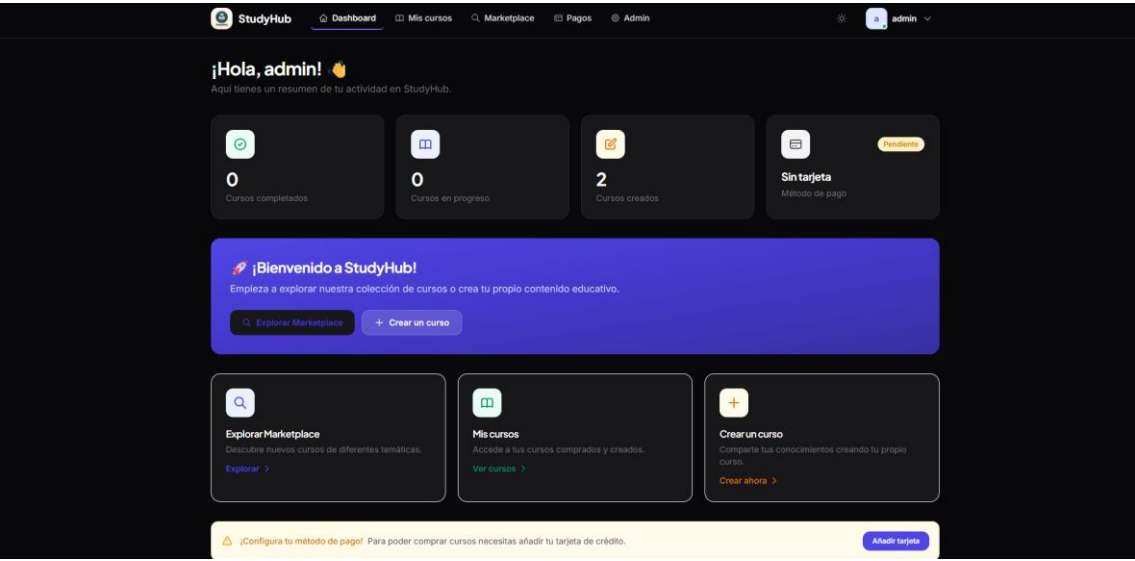
POLÍTICA DE PRIVACIDAD:



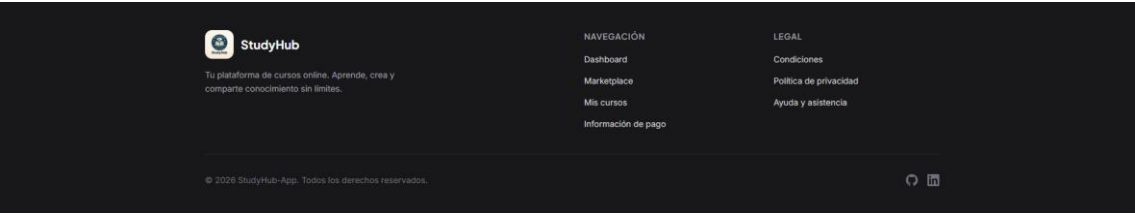
FAQS CON BUSCADOR EN TIEMPO REAL:



MODO OSCURO TAMBIÉN DISPONIBLE



FOOTER COMÚN



*Todo lo que aparece en la página es FUNCIONAL.

4. Construcción del Sistema

4.1. GitHub

Para el desarrollo colaborativo y el control de versiones, se ha hecho uso de un repositorio en **GitHub**. Todo el código fuente, la documentación técnica y el histórico de cambios residen de forma centralizada en dicho repositorio.

Estrategia de Ramificación (Git Flow adaptado):

- **Rama `main`**: Representa la versión estable del proyecto lista para producción. Solo recibe actualizaciones (merges) cuando una versión es completamente funcional.
- **Rama `develop`**: Es la rama principal de integración. Todo el código nuevo se sube aquí para probar que las diferentes partes interactúan correctamente antes de pasar a `main`.
- **Ramas de funcionalidad (`feature/`)**: Para cada nueva pantalla, servicio o componente se ha creado una rama independiente, garantizando un desarrollo modular y evitando conflictos.
- **Control y Seguimiento**: Se han registrado commits atómicos y descriptivos que permiten trazar la evolución temporal del TFG y justificar las horas invertidas.

Enlace al repositorio:

<https://github.com/manumengo71/StudyHub-App>

4.2. Docker

Con el objetivo de aislar el entorno de desarrollo y facilitar el despliegue tanto en local como en producción, la aplicación está completamente "dockerizada". Esto asegura que "funcione igual en todas las máquinas", solucionando el clásico problema de incompatibilidad de versiones locales.

Mediante el archivo `docker-compose.yml`, el sistema levanta los siguientes servicios interconectados:

- **`studyhub-app`**: Contenedor principal con Apache y PHP 8.2. Incluye un script `docker-entypoint.sh` que se encarga automáticamente de instalar dependencias, dar permisos a carpetas y ejecutar migraciones en el arranque.

- **`studyhub-db`**: Contenedor de MySQL 8.0 donde residen los datos, los cuales persisten gracias al mapeo de volúmenes de Docker.
- **`mailpit`**: Servidor SMTP virtual utilizado para interceptar los correos de registro y restablecimiento de contraseñas de Laravel durante la fase de pruebas, sin necesidad de enviar correos reales.
- **`phpmyadmin`**: (Opcional) Interfaz web para facilitar consultas directas a la base de datos sin entrar a la consola.

5. Conclusiones

El desarrollo de **StudyHub-App** ha supuesto un reto técnico y de diseño integral que culmina de manera satisfactoria. La plataforma cumple con creces los objetivos planteados inicialmente: se ha logrado un producto funcional, de estética premium y altamente escalable.

A nivel técnico, la integración de **Laravel 10** con **Tailwind CSS** ha demostrado ser una combinación excepcional para el desarrollo de aplicaciones SaaS rápidas y mantenibles. El uso de patrones de diseño, como la separación de lógica en una capa de *Servicios* y el uso de *Form Requests* para validación, han dado como resultado un código limpio. La implementación de bibliotecas como *Spatie Permissions* o *DomPDF* han añadido capas de complejidad profesional (gestión de roles y generación de facturas/certificados) que elevan el valor del sistema.

Desde el punto de vista del aprendizaje personal, el proyecto ha permitido consolidar conocimientos de desarrollo Full-Stack, fortalecer el diseño de bases de datos relacionales, y comprender la importancia de las herramientas de DevOps (Docker, Git) en el ciclo de vida del software moderno.

6. Glosario de términos

- **MVC (Modelo-Vista-Controlador):** Patrón de arquitectura de software que separa los datos (Modelo), la interfaz de usuario (Vista) y la lógica de control (Controlador).
- **ORM (Object-Relational Mapping):** Técnica de programación para convertir datos entre el sistema de tipos de una base de datos relacional y objetos en lenguajes de programación orientados a objetos (ej. *Eloquent* en Laravel).
- **SaaS (Software as a Service):** Modelo de distribución de software donde el soporte lógico y los datos se alojan en servidores de un proveedor TIC, y se accede a ellos vía web.
- **Blade:** Motor de plantillas simple pero potente proporcionado por Laravel.
- **Tailwind CSS:** Framework CSS "utility-first" que permite crear diseños rápidos y personalizados directamente desde el HTML.
- **Vite:** *Herramienta de compilación para el frontend que proporciona un entorno de desarrollo ultrarrápido y empaquetado optimizado.

7. Bibliografía y Webgrafía

- **Documentación oficial de Laravel (v10.x):** <https://laravel.com/docs/10.x>
- **Documentación oficial de Tailwind CSS:** <https://tailwindcss.com/docs>
- **Documentación de Spatie Laravel Permission:** <https://spatie.be/docs/laravel-permission>
- **Documentación de Docker y Docker Compose:** <https://docs.docker.com>
- **Alpine.js:** <https://alpinejs.dev/>

Apéndices

Apéndice A. Instrucciones de instalación local (Guía Rápida)

Para reproducir el entorno en una máquina de evaluación, se deben ejecutar los siguientes pasos en una terminal:

- Clonar repositorio: ``git clone https://github.com/manumengo71/StudyHub-App``
- Desde la raíz del proyecto, ejecuta el comando para construir y levantar los contenedores en segundo plano: powershell
`docker-compose up --build -d`
- Una vez que los contenedores estén corriendo, entra de manera interactiva al contenedor de la aplicación: powershell
`docker exec -it mi-contenedor-laravel /bin/bash`
- Dentro del contenedor, ejecuta los comandos de Artisan:
 - A. Crear las tablas: `php artisan migrate`
 - B. Poblar con datos (Elegir una opción)
 - Opción con datos de prueba (Recomendada para enseñar la web llena):
`php artisan db:seed`
 - Opción "Limpia" (Solo lo mínimo para funcionar):
`php artisan db:seed --class=DatabaseProductionSeeder`
- Una vez terminados los comandos, puedes abrir el navegador en:
 - Web: `http://localhost:8000`
 - Base de datos (phpMyAdmin): `http://localhost:8080`
 - Correos (Mailpit): `http://localhost:8025` (Si está configurado)

Apéndice B. Cuadro de Revisiones de la Documentación

Fecha	Descripción del cambio	Responsable
 Fase Inicial	Definición del Estudio de Viabilidad y Requisitos	Manuel C. Mendoza González
Fase Media	Incorporación del Análisis del Sistema e Interfaces	Manuel C. Mendoza González
Fase Final	Cierre de manuales, Dockerización y Conclusiones	Manuel C. Mendoza González